Automated IT Skills Assessment and Performance Evaluation System

Final Year Project Proposal

Student	Mudassar Hussain
Registration No.	F23BINFT1M01117
Supervisor	Dr Suleman Shah
Department / University	DIT Islamia University of Bahawalpur
Degree Program	BS Information Technology (7th Semester)
Academic Session	2023-27
Submission Date	October 2026

1. Abstract

Students who want to check their basic IT skills usually end up using a mix of quiz websites, typing sites and class tests. Each one gives its own separate result, and none of them keeps a record the student can look back on. It is hard to say which topics are really weak, or whether practice is actually helping.

This project proposes a web-based software where a student can take assessments in computer fundamentals, networking, SQL and database concepts, typing, and introductory programming, all in one place. The system will score supported question types automatically, give feedback for each category, point out the weaker areas and keep a history of every attempt. Students will see their results on a personal dashboard, while an administrator manages the questions, categories and user records.

The first version will use clear, rule-based scoring and simple descriptive statistics. It will not claim any artificial intelligence features that the project cannot honestly support. The platform is meant for practice and self-evaluation. It is not a replacement for university exams or professional certification.

2. Introduction and Background

IT students need theory as well as hands-on familiarity with the basics: how computers and networks work, how to write a simple database query, how to type reasonably fast and accurately, and how to read a short program. In practice these skills get tested in different places. A teacher may run a paper quiz, a student may try an online typing test, and SQL practice may happen on yet another website. The results end up scattered, and nobody has one view of progress over a semester.

A single platform can fix part of this. If the questions, scoring rules and results live in one system, a student can see how they did in each area, compare today's attempt with last month's, and decide what to practise next. Administrators benefit as well, since the question bank sits in one place and can be reviewed and corrected over time.

There is also a question of trust. A score is only useful if the student can see how it was worked out. For that reason this project keeps its scoring simple and visible. Each result will show the marks earned, the marks available and the rule behind the feedback label.

3. Problem Statement

Students currently lack a single, easy-to-reach platform that can:

- Assess several foundational IT skills using the same scoring rules.
- Show category-wise results so weak areas are easy to spot.
- Keep a history of attempts so improvement can be tracked over time.
- Give feedback that explains how a result was reached.
- Let an administrator maintain questions and users without touching the code.

4. Aim and Objectives

Aim: To design, build and test a web-based system that lets students assess selected IT skills, receive automated results and feedback, identify areas that need practice, and track their performance over time.

Objectives:

- Develop a secure, easy-to-use web application with student login and an administrator role.
- Provide assessment categories for computer fundamentals, networking, SQL/database concepts, typing and introductory programming.
- Evaluate objective questions and supported exercise types automatically using defined rules.
- Calculate total scores and category-wise performance summaries.
- Identify comparatively strong and weak areas and give feedback a student can understand.
- Store every assessment attempt so students can compare results over time.
- Give administrators tools to manage categories and questions.
- Evaluate functional correctness, scoring reliability, usability and basic security.

5. Scope

In scope

- Responsive web application for students and one administrator role, usable on desktop and mobile screens.
- Registration, login and a personal dashboard for each student.
- Five assessment categories with timed quizzes and automatic scoring of objective questions.
- A typing test that reports words per minute and accuracy.
- SQL exercises checked against expected answers or result sets, where this is practical.
- Introductory programming questions based on concepts and output prediction.

- Result summaries, category feedback, attempt history and progress charts.
- Administrator tools for the question bank, categories and user records.
- Automated tests for scoring, typing metrics and access control.

Out of scope (possible future work)

- Formal examinations, certification or anything that replaces university assessment.
- Running arbitrary student code on the main server. Code execution will only be considered if a safe, isolated runner turns out to be feasible.
- Machine-learning or AI-based grading, and adaptive question selection.
- Exam proctoring, webcam monitoring or plagiarism detection.
- Grading of essays or long written answers.
- Native mobile apps and integration with university student information systems.

6. Proposed System Overview

6.1 Main user flows

- **Sign in:** the student registers or logs in and lands on a personal dashboard.
- **Choose an assessment:** the student picks a category and reads the instructions and time limit.
- **Take the test:** the student answers the questions, or types the passage in the typing test, while a timer stays visible.
- **Get results:** the system scores the attempt and shows the total, the category-wise scores and, where suitable, the correct answers with short explanations.
- **Review progress:** the dashboard lists earlier attempts, shows charts and highlights weaker categories.
- **Manage content (administrator):** the administrator adds, edits or disables questions and categories, sets time limits and manages user records.

6.2 Key features

Area	Features
Accounts	Registration, login, hashed passwords, personal dashboard
Assessments	Category-based tests, instructions, time limits, submission when time runs out
Fundamentals and networking	Multiple-choice and true/false questions, automatic scoring, explanations after submission
SQL / database	Exercises checked against predefined answers or expected result sets where practical
Programming	Concept and output-prediction questions; code execution only with safe isolation
Typing	Elapsed time, words per minute and character accuracy

Area	Features
Results	Total score, category scores, feedback label and the rule behind it
Progress	Attempt history, comparison with earlier scores, charts
Administration	Question bank, categories, assessment settings and user records
Usability	Responsive layout, keyboard-friendly navigation, clear messages

7. System Design

7.1 Architecture

The system follows a simple layered design, with the scoring logic kept separate from the pages that display it.

Layer	Responsibility
Presentation	Browser interface for login, assessments, results and dashboards
Application	Flask routes, authentication, test delivery, scoring, feedback and reporting
Assessment services	Question evaluation, typing metrics, SQL checks and, if feasible, isolated code evaluation
Data layer	MySQL tables for users, categories, questions, attempts, answers, scores and feedback
Administration	Role-protected screens for managing users, questions, categories and assessment content

7.2 User roles

Role	Permissions
Student	Take assessments, view personal results and review their own attempt history
Administrator	Manage categories, questions, assessment settings and relevant user records

7.3 Technology stack (with rationale)

Component	Choice	Why
Frontend	HTML5, CSS3, JavaScript, Bootstrap	Gives a responsive layout without a lot of custom CSS
Backend	Python with Flask	Lightweight, and keeps scoring logic in plain Python that is easy to test
Database	MySQL	Relational, and a good fit for users, attempts and responses
Charts	Chart.js	Simple browser-side charts for category scores and progress
Development	Visual Studio Code	Familiar editor with good debugging support

Component	Choice	Why
Version control	Git and GitHub	Source history, backup and a record of progress for the supervisor
Testing	Pytest (or equivalent) plus manual tests	Checks scoring logic and full workflows
Deployment	Cloud host or university/local server	Needed for the final demonstration

7.4 Assessment and scoring engine

Every assessment type has its own evaluation rule, and the rules are written down so they can be checked by hand.

Area	Evaluation approach
Computer fundamentals	Objective questions scored automatically
Networking	Objective questions on selected networking concepts
SQL / database	Exercises checked against predefined answers or expected result sets where practical
Typing	Elapsed time, typed words, words per minute and character accuracy using fixed formulas
Introductory programming	Starts with concept and output-prediction questions; execution only with safe isolation

Typing formulas. Words per minute will be worked out with the usual convention of five characters per word: WPM = (correct characters ÷ 5) ÷ minutes taken. Accuracy = correct characters ÷ total characters typed × 100.

Feedback logic. Feedback labels come from documented score thresholds. As a starting point I plan to use “Needs Practice” below 50%, “Developing” from 50% to 74% and “Strong” from 75%, with the cut-offs configurable and agreed with the supervisor. These labels describe one completed attempt. They are not a final judgement of anyone’s ability.

What each result will show:

- Marks earned out of total marks available.
- Performance in each category.
- The rule that produced the feedback label.
- Correct answers or short explanations after submission, where suitable.
- A short note that the result reflects the chosen question bank and attempt only.

7.5 Data model (core entities)

The schema will be refined during the design phase. The core entities are:

Entity	Fields / purpose
Users	user_id, name, email, password_hash, role, created_at
Categories	category_id, name, description, active_status

Entity	Fields / purpose
Assessments	assessment_id, category_id, title, time_limit, status
Questions	question_id, assessment_id, question_type, prompt, options/reference data, marks
Attempts	attempt_id, user_id, assessment_id, started_at, submitted_at, total_score, status
Responses	response_id, attempt_id, question_id, submitted_response, awarded_marks, correctness
TypingResults	attempt_id, elapsed_time, words_per_minute, accuracy_percent (typing attempts only)
Feedback	feedback_id, attempt_id, category_id, feedback_label, explanation

One user can have many attempts, one category can hold many assessments, one assessment can hold many questions and one attempt can hold many responses. Passwords will be stored only as secure hashes, never as plain text.

8. Responsible Design: Privacy, Security and Fairness

8.1 Privacy and data handling

- Collect only what is needed for accounts, assessments and progress tracking.
- Explain what is stored and why, and avoid sensitive information that has no use here.
- Provide a reasonable way to correct or delete account data, within what the university allows.
- Use anonymized or aggregated results in demonstrations unless users have agreed to be identified.

8.2 Security

- Secure password hashing and role-based access control.
- Input validation and parameterized database queries, with protection against common web risks such as those listed by OWASP.
- Personal results are private: one student must never be able to open another student's data.
- No arbitrary code runs on the main server. Programming questions will either use a restricted, isolated environment or stay non-executable.

8.3 Fairness

- Clear, consistent scoring rules for everyone taking the same version of an assessment.
- Questions are reviewed for ambiguous wording, and no personal or demographic details play any part in scoring.
- Students can report a confusing or faulty question so it can be reviewed.
- Results are presented as a reflection of the selected question bank and one attempt, not a full measure of IT competence.

8.4 Accessibility and usability

- Readable typography, enough contrast, clear labels and consistent navigation.
- Results never rely on colour alone. Colour is always paired with text and scores.
- Responsive layouts, and keyboard support for the main workflows.
- Plain instructions and understandable error and validation messages.

9. Methodology

The project follows an iterative approach, so that each feature can be built, demonstrated, tested and corrected in small stages instead of all at the end.

- **Requirements analysis:** confirm user roles, categories, question types, scoring rules and scope with the supervisor.
- **System design:** use-case and activity diagrams, database schema, interface wireframes and security assumptions.
- **Implementation:** build authentication, assessment management, test delivery, scoring, feedback and progress tracking step by step.
- **Integration:** connect the frontend, the Flask services and the MySQL database.
- **Validation:** test expected behaviour, scoring formulas, access control and usability with sample data.
- **Deployment and documentation:** prepare the demo build, user guide, test evidence and final report.

10. Testing and Evaluation Plan

10.1 Software testing

Level	Scope
Unit	Scoring functions, typing metrics, input validation and feedback thresholds
Integration	Frontend and backend requests, database writes, submissions and result generation
System	Complete student and administrator workflows, from login to results
Security	Role access, password handling, input validation, sessions and private results
Usability	Whether users can follow instructions, finish tests and read the results
Compatibility	Core workflows on supported desktop and mobile browsers

10.2 Evaluation of the scoring engine

This project does not use a machine-learning model, so evaluation focuses on whether the assessment engine is correct and consistent.

- **Scoring:** compare system scores with scores calculated by hand.
- **Typing:** check WPM and accuracy using fixed passages and known times.

- **SQL:** compare submitted results with the predefined expected output for supported exercises.
- **Feedback:** confirm labels and explanations follow the configured thresholds.
- **Questions:** review clarity, answer keys, marks and category coverage.
- **Reports:** confirm saved results match the submitted attempt and display correctly.

10.3 Success criteria

- All critical workflows pass the planned functional tests.
- Scoring functions match the expected results for the prepared test cases.
- Students can reach only their own results, and administrator functions need administrator permission.
- Attempts and results are stored and retrieved consistently.
- Test users can complete the main workflow and understand their result summary. Problems found are documented and fixed where feasible.

11. Project Timeline (32 Weeks)

This schedule is a proposal and will be adjusted to the department's academic calendar and the supervisor's milestones.

Phase	Weeks	Activities	Deliverable
1. Topic and requirements	1–3	Confirm scope, stakeholders, objectives and requirements	Approved requirements and scope
2. Background study	4–5	Review assessment systems, scoring approaches and technical documentation	Background review and references
3. Analysis and design	6–8	Use cases, workflows, database model, UI wireframes	Diagrams, schema and interface plan
4. Environment setup	9–10	Repository, Flask app, database, base UI	Working project skeleton
5. Authentication and admin	11–13	Login, roles, profiles, categories, question management	Authentication and admin module
6. Assessment delivery	14–17	Assessment pages, navigation, timers, submission	Working assessment workflow
7. Evaluation engine	18–21	Objective scoring, typing metrics, feasible SQL checks	Scoring engine and sample tests
8. Results and progress	22–24	Feedback rules, dashboard, attempt history, charts	Results and progress dashboard
9. Integration	25–26	Connect modules and fix workflow defects	Integrated beta version
10. Testing	27–29	Functional, security, scoring, compatibility and usability tests	Test cases, results, issue log

Phase	Weeks	Activities	Deliverable
11. Deployment and documentation	30–31	Deployment, user guide, technical documents, report	Deployable build and report draft
12. Final review	32	Supervisor feedback and demo preparation	Final build, report, presentation

12. Risks and Mitigation

Risk	Impact	Mitigation
Scope creep	Core features may stay unfinished	Prioritize core features; treat code execution as optional
Unsafe code execution	Server or data compromise	Never run arbitrary code on the main server; use a sandbox or non-executable questions
Incorrect scoring	Misleading results	Prepare known-answer test cases and verify the formulas
Weak question bank	Results may not reflect the intended skills	Review questions, answer keys and category coverage
Data exposure	Student results seen by the wrong person	Password hashing, access control, validation and limited data collection
Schedule delays	Testing or documentation rushed	Track progress weekly and review milestones with the supervisor
Usability barriers	Students misread tests or results	Clear instructions, accessible design and feedback from test users

13. Expected Outcomes and Deliverables

- A responsive web-based IT skills assessment application.
- Student authentication and an administrator dashboard.
- Assessment and question management for the agreed categories.
- Automated scoring for supported question types and typing metrics.
- SQL exercises with predefined evaluation where practical.
- A results dashboard with category-wise feedback and previous-attempt tracking.
- A relational database schema and documented architecture.
- Source code in a GitHub repository.
- Test plan, test cases, evaluation evidence and documented limitations.
- User guide, technical documentation, final report and a live demonstration.

14. Significance and Contribution

The practical value of this project is in bringing several basic IT self-assessments into one platform with the same scoring rules and a saved history. Instead of handing the student one final number, the system breaks performance down by skill category and shows the rule behind its feedback. That

helps a student decide which topics need more practice, and it gives administrators a structured question bank to maintain. The value of the work will be shown through the working system, checked scoring behaviour, feedback from test users and an honest list of its limitations.

15. Next Steps

1. Submit the proposal and confirm the title, scope and categories with the supervisor.
2. Agree which exercise types must be in the first working version.
3. Prepare the final requirements, use-case diagram and database ER diagram.
4. Create the GitHub repository with a README describing the project, stack and modules.
5. Build a proof of concept with login, one quiz, automatic scoring and saved results.
6. Expand step by step along the 32-week plan and show progress at supervisor checkpoints.

16. References

These are starting points. The final report will cite only the sources I actually use, along with academic literature found during the background study.

- Python Software Foundation. *Python Documentation.* https://docs.python.org/3/
- Pallets Projects. *Flask Documentation.* https://flask.palletsprojects.com/
- MySQL. *MySQL Reference Manual.* https://dev.mysql.com/doc/
- Bootstrap. *Documentation.* https://getbootstrap.com/docs/
- Chart.js. *Documentation.* https://www.chartjs.org/docs/
- OWASP Foundation. *OWASP Top 10: Web Application Security Risks.* https://owasp.org/www-project-top-ten/